

Практическая работа № 1(3)

Применение различных технологий размещения элементов в HTML-документе

Цель: изучить возможности верстки с использованием модуля CSS Grid Layout.

Сеточная разметка

Grid-раскладка

Grid-раскладка — новая концепция, ещё более гибкая чем flex, позволяющая при блочной вёрстке управлять положением и размером блоков макета ещё гибче и легче.

Стилевое свойство контейнера **display** со значением **grid** превращает его в **grid-контейнер**, т.е. контейнер, изменяющий положение и размеры своих элементов (grid-элементов) по своим правилам.

Grid-контейнер напоминает сетку (таблицу), т.е. состоит из **grid-строк**, **grid-столбцов** и **grid-ячеек**. Grid-строки и grid-столбцы называются **треками**. В grid-ячейках или группах grid-ячеек можно размещать произвольное содержимое — будем называть такое содержимое **grid-элементами**. Grid-элементами автоматически становятся все потомки (дочерние теги) grid-контейнера.

Стилевые свойства grid-контейнера **grid-template-columns** и **grid-template-rows** позволяют задать, сколько grid-строк и grid-столбцов в нём будет и каковы их размеры.

HTML-код
<pre><div style="display: grid; border: solid red 1px; grid-template-columns: 100px 150px 100px; grid-template-rows: 30px 20px"> <div style="background-color: magenta">AAA</div> <div style="background-color: cyan">BBB</div> <div style="background-color: silver">CCC</div> <div style="background-color: yellow">DDD</div> <div style="background-color: red">EEE</div> <div style="background-color: green">FFF</div> </div></pre>
Отображение в браузере


На этом примере видно, что grid-элементы (потомки grid-контейнера) автоматически распределились по grid-ячейкам слева направо и сверху вниз. Другие способы автоматического размещения можно задать стилевым свойством **grid-auto-flow**, но мы это рассматривать не будем, т.к. основной вариант размещения grid-элементов — это как раз прямое указание grid-ячеек, в которых их нужно разместить (рассмотрим это ниже).

Также на этом примере видно, что grid-контейнер не занял всю доступную ему ширину, т.к. мы указали точные ширины колонок, в сумме меньшие, чем ширина страницы. Положение grid-контейнера внутри доступного ему пространства можно задавать стилевыми свойствами **justify-content** и **align-content** grid-контейнера. Для указания размеров треков (grid-колонок и grid-строк) можно использовать любые привычные единицы размеров (например, проценты — 50% это будет половина доступной ширины).

Но есть и ещё одна, новая и очень важная единица размера — **fr**:

HTML-код
<pre><div style="display: grid; border: solid red 1px; grid-template-columns: 1fr 100px 2fr; grid-template-rows: 30px 20px"> <div style="background-color: magenta">AAA</div> <div style="background-color: cyan">BBB</div> <div style="background-color: silver">CCC</div> <div style="background-color: yellow">DDD</div> <div style="background-color: red">EEE</div> <div style="background-color: green">FFF</div> </div></pre>
Отображение в браузере


Смысл таков — вторая grid-колонка должна быть шириной 100px, а **всё оставшееся место** нужно поделить между первой и третьей grid-колонками, причём третьей grid-колонке должно достаться вдвое больше чем первой. Аналогично управляются размеры grid-строк.

Если grid-колонок или grid-строк много, чтобы не повторять их размеры, можно воспользоваться синтаксисом `repeat`:

HTML-код
<pre><div style="display: grid; border: solid red 1px; grid-template-columns: 100px repeat(2, 50px 1fr); grid-template-rows: 30px 20px"> <div style="background-color: magenta">AAA</div> <div style="background-color: cyan">BBB</div> <div style="background-color: silver">CCC</div> <div style="background-color: yellow">DDD</div> <div style="background-color: red">EEE</div> <div style="background-color: green">FFF</div> <div style="background-color: gray">GGG</div> </div></pre>
Отображение в браузере


Запись `repeat(2, 50px 1fr)` означает — два раза повторить комбинацию ширин grid-колонок 50px и 1fr, т.е. запись `grid-template-columns: 100px repeat(2, 50px 1fr)` аналогична записи `grid-template-columns: 100px 50px 1fr 50px 1fr`.

На этом примере также видно, что если grid-элементов меньше чем grid-ячеек, остаются незаполненные grid-ячейки.

Ширину grid-колонки или высоту grid-строки можно также задать в виде **minmax(100px, 200px)** — это значит минимум 100px, максимум 200px. Например, запись `minmax(100px, auto)` означает — размер должен быть не менее 100px и не более размера содержимого grid-ячеек.

Grid-элемент может занимать и несколько grid-ячеек сразу.

Между grid-ячейками есть (воображаемые) **grid-линии**. Например, если в grid-контейнере 2 grid-колонки, то слева, справа и между ними есть 3 вертикальные grid-линии. Если в grid-контейнере 3 grid-строки, то выше, ниже и между ними есть 4 горизонтальные grid-линии.

Прямоугольные области из нескольких grid-ячеек называются **grid-области**.

Для любого из grid-элементов мы можем явно указать, в какую из grid-областей его надо разместить. Grid-область задаётся grid-линиями, которые её ограничивают.

HTML-код	
<pre><div style="display: grid; border: solid red 1px; grid-template-columns: 200px 1fr 150px; grid-template-rows: 40px 40px 40px"> <div style="background-color: magenta; grid-column: 2 / 4; grid-row: 1">AAA</div> <div style="background-color: silver; grid-column: 1; grid-row: 2 / 4">BBB</div> <div style="background-color: yellow; grid-column: 2; grid-row: 2">CCC</div> </div></pre>	
Отображение в браузере	
	

Здесь элемент AAA занимает по горизонтали между 2-й и 4-й вертикальными grid-линиями, элемент В занимает по вертикали место между 2-й и 4-й горизонтальными grid-линиями.

Добиться, чтобы grid-элемент занимал сразу несколько grid-ячеек, можно также ключевым словом **span**:

HTML-код	
<pre><div style="display: grid; border: solid red 1px; grid-template-columns: 200px 1fr 150px; grid-template-rows: 40px 40px 40px"> <div style="background-color: magenta; grid-column: 2; grid-row: 1">AAA</div> <div style="background-color: silver; grid-column: 3; grid-row: 1 / span 2">BBB</div> <div style="background-color: yellow; grid-column: 1 / span 2; grid-row: 2 / span 2">CCC</div> </div></pre>	
Отображение в браузере	
	

Т.е. элемент BBB начинается с 1-й горизонтальной grid-линии и занимает по высоте 2 grid-строки, элемент CCC расположен ниже 2-й горизонтальной grid-линии и правее 1-й вертикальной grid-линии и занимает по ширине две grid-колонки и по высоте две grid-строки.

Можно, указывая положение и/или размер grid-элемента, «вылезти» за пределы grid-контейнера. При этом автоматически создаются т.н. **неявные** grid-колонок и grid-строки, и их размер можно задать стилевыми свойствами **grid-auto-columns** и **grid-auto-rows**.

HTML-код
<pre><div style="display: grid; border: solid red 1px; grid-template-columns: 150px 150px 150px; grid-template-rows: 40px 40px 40px; grid-auto-columns: 50px"> <div style="background-color: magenta; grid-column: 3; grid-row: 1">AAA</div> <div style="background-color: silver; grid-column: 1; grid-row: 2 / 4">BBB</div> <div style="background-color: green; grid-column: 2 / span 3; grid-row: 2">CCC</div> </div></pre>
Отображение в браузере


Здесь в гриде задано 3 колонки с точно заданной шириной 150px, и элемент CCC начинается со 2-й вертикальной grid-линии и занимает ширину 3 колонок, хотя доступно только две. Браузер автоматически создаёт для размещения элемента CCC ещё одну колонку шириной 50px.

Не всегда удобно оперировать номерами grid-колонок. Им можно задавать символические имена, прямо в стилевых свойствах grid-template-columns и grid-template-rows:

HTML-код
<pre><div style="display: grid; border: solid red 1px; grid-template-columns: 150px [center] 150px 150px; grid-template-rows: 40px 40px 40px [bottom]; grid-auto-columns: 50px"> <div style="background-color: magenta; grid-column: 3; grid-row: 1">AAA</div> <div style="background-color: silver; grid-column: 1; grid-row: 2 / bottom">BBB</div> <div style="background-color: green; grid-column: center / span 3; grid-row: 2">CCC</div> </div></pre>
Отображение в браузере

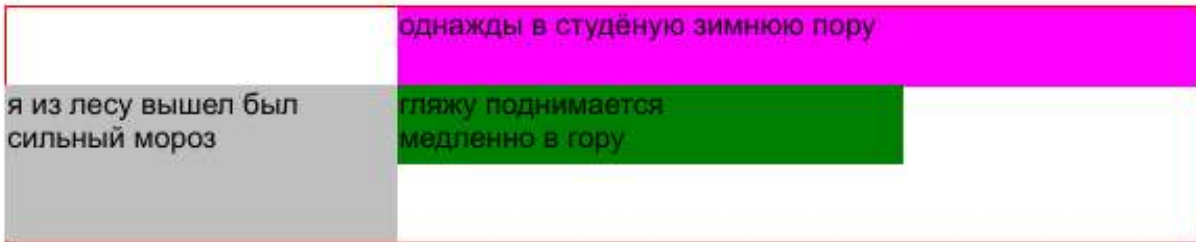

Здесь 2-й вертикальной grid-линии (т.е. расположенной между 1-й и 2-й grid-колонками) дано имя center, а 4-й горизонтальной grid-линии (т.е. расположенной после 3-й grid-строки) дано имя bottom.

Ширину зазора между grid-столбцами и grid-строками можно задать стилевыми свойствами **grid-column-gap** и **grid-row-gap**:

HTML-код	
<pre><div style="display: grid; border: solid red 1px; grid-template-columns: 200px 1fr 150px; grid-template-rows: 40px 40px 40px; grid-column-gap: 10px; grid-row-gap: 5px;"> <div style="background-color: magenta; grid-column: 2 / 4; grid-row: 1">AAA</div> <div style="background-color: silver; grid-column: 1; grid-row: 2 / 4">BBB</div> <div style="background-color: green; grid-column: 2; grid-row: 2">CCC</div> </div></pre>	
Отображение в браузере	
	

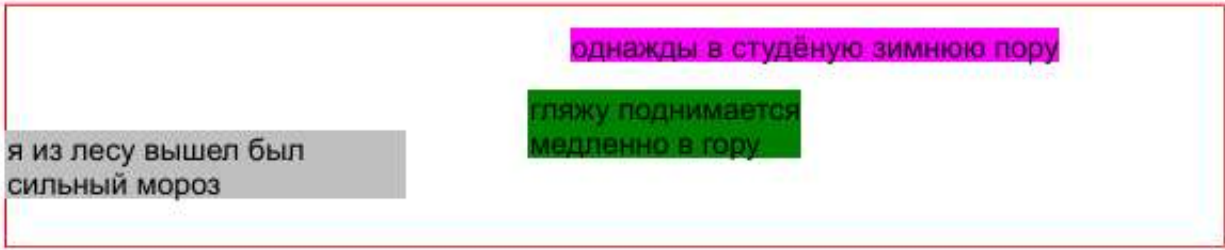
Эти зазоры не являются grid-линиями, не путайте их.

По умолчанию всё содержимое grid-ячеек (и grid-областей) растягивается на всю доступную площадь ячейки (области) и выравнивается к левому верхнему углу:

HTML-код	
<pre><div style="display: grid; border: solid red 1px; grid-template-columns: 200px 1fr 150px; grid-template-rows: 40px 40px 40px;"> <div style="background-color: magenta; grid-column: 2 / 4; grid-row: 1"> однажды в студёную зимнюю пору</div> <div style="background-color: silver; grid-column: 1; grid-row: 2 / 4"> я из лесу вышел был сильный мороз</div> <div style="background-color: green; grid-column: 2; grid-row: 2"> гляжу поднимается
медленно в гору</div> </div></pre>	
Отображение в браузере	
	

Стилевые свойства **justify-items** и **align-items**, применяемые к grid-контейнеру, позволяют содержимое grid-ячеек и grid-областей:

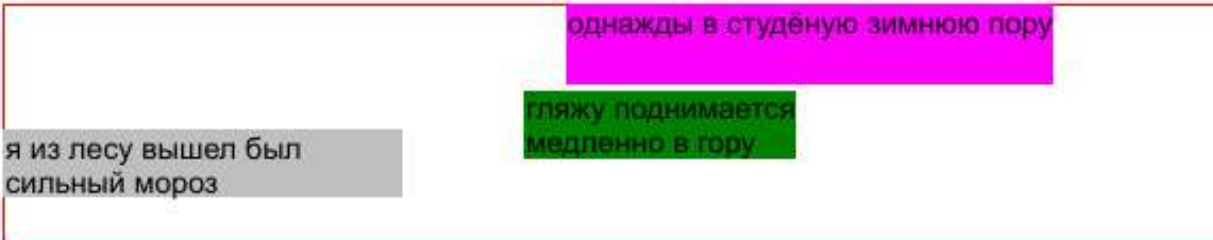
- **start** — выровнять влево и вверх соответственно;
- **end** — выровнять вправо и вниз соответственно;
- **center** — отцентрировать;
- **stretch** — растянуть на всю ширину и высоту соответственно (это значение по умолчанию).

HTML-код
<pre><div style="display: grid; border: solid red 1px; grid-template-columns: 200px 1fr 150px; grid-template-rows: 40px 40px 40px; justify-items: center; align-items: center;"> <div style="background-color: magenta; grid-column: 2 / 4; grid-row: 1"> однажды в студёную зимнюю пору</div> <div style="background-color: silver; grid-column: 1; grid-row: 2 / 4"> я из лесу вышел был сильный мороз</div> <div style="background-color: green; grid-column: 2; grid-row: 2"> гляжу поднимается
медленно в гору</div> </div></pre>
Отображение в браузере


На вкладке Elements в Chrome очень хорошо видно, что содержимое grid-областей теперь не растягивается, а центрируется:

	однажды в студёную зимнюю пору	
я из лесу вышел был сильный мороз	гляжу поднимается медленно в гору	

Упомянутые выше стилевые свойства **justify-items** и **align-items** применяются к grid-контейнеру и действуют на все grid-элементы, а стилевые свойства **justify-self** и **align-self** (с такими же возможными значениями — **start, end, center, stretch**) применяются к конкретному grid-элементу и позволяют выровнять его как-либо иначе:

HTML-код
<pre><div style="display: grid; border: solid red 1px; grid-template-columns: 200px 1fr 150px; grid-template-rows: 40px 40px 40px; justify-items: center; align-items: center;"> <div style="background-color: magenta; grid-column: 2 / 4; grid-row: 1; align-self: stretch"> однажды в студёную зимнюю пору</div> <div style="background-color: silver; grid-column: 1; grid-row: 2 / 4"> я из лесу вышел был сильный мороз</div> <div style="background-color: green; grid-column: 2; grid-row: 2"> гляжу поднимается
медленно в гору</div> </div></pre>
Отображение в браузере


Здесь розовая ячейка по горизонтали по-прежнему не растягивается а центрируется, а по вертикали — растягивается на всю высоту grid-ячейки, т.к. ей задан стиль **align-self: stretch**.

Grid-раскладку поддерживают не все браузеры:

Chrome 57+, Firefox 52+, Opera 44+, IE 10+ и Edge 12+ (старый синтаксис), Edge 16+, Opera 44+, Safari 10.1+, iOS Safari 10.3+, Android Browser 76+

Порядок выполнения работы

1. Протестируйте рассмотренные в лабораторной работе примеры.
2. Запишите в тетради определение модуля Grid Layout, назначение сеточной верстки, основных ее понятий и особенностей [1, 178-202].
3. Изучите значения свойств grid-template-columns, свойств grid-template-rows, свойств grid-template-areas, свойств grid-template, grid-row-start, grid-row-end, grid-column-start, grid-column-end, justify-items, align-items, justify-content, align-content, grid-auto-flow, grid по таблицам 57-64 [1, 181-200].
4. Разберитесь и отладьте пример кода веб-документа сеточной верстки с медиазапросами [1, 211-218].
5. Закрепите основные понятия сеточной верстки на следующих ресурсах:
 - https://developer.mozilla.org/ru/docs/Web/CSS/Guides/Grid_layout/Basic_concepts - статья (основы)
 - <https://tuhub.ru/posts/css-grid-complete-guide> - статья (более подробно)
 - <https://css-live.ru/css/bolshaya-statya-pro-gridy-css-grid-layout.html> - статья (хорошо и подробно);
 - <https://cssgridgarden.com/#ru> - обучающая игра по Grid Layout.
6. Оформите отчёт.
7. Защитите лабораторную работу.

Домашнее задание

8. Создайте адаптивный многостраничный сайт на основе сеточной верстки (за образец рекомендуется взять пример из учебника) [1, 211 – 213]. Содержание страниц наполнить выполненными заданиями из практической работы.
10. Письменно ответить на контрольные вопросы [1, 202].

Литературные источники

1. Брылёва, А. А. Программные средства создания интернет-приложений : учеб. Пособие / А. А. Брылёва. – Минск : РИПО, 2022. – 483 с. : ил.